IT305 COMPUTER NETWORKS LABORATORY

Department of Information and Communication Technology
Dhirubhai Ambani Institute of Information and Communication Technology (DA-IICT)

Fault-Tolerant Topic-Based File Distribution Service

Comprehensive Design, Socket Protocol Implementation, and Empirical Evaluation

Merged Project Groups: Group 071 & Group 072

Academic Semester: Autumn 2026 | Submission Date: October 21, 2026

Team Members and Individual Contributions

Member Name	Roll Number	Specific Individual Contribution Statement
Jay Vaghela	202401237	Architected Part 1 sequential and multi-threaded server core; implemented client-server socket lifecycle, worker thread dispatching, and Part 1 concurrency scaling experiments.
Dharmraj Vaghela	202401236	Designed Part 2 Case 2 checkpoint-resume protocol, server-authoritative session state table with thread-safe mutex synchronization, and wire telemetry instrumentation.
Vivek Dodiya	202401253	Implemented Part 2 Case 1 full-restart recovery mechanism, client-side directory purging, file truncation logic, and fault injection verification harnesses.
Atharv Trivedi	202401230	Architected Part 2 Case 2 Enhanced parallel connection engine, dynamic work unit planning, chunk slicing, and multi-connection file reassembly using `pwrite()`.
Shlok Topaliya	202401229	Developed automated regression test suites, supplementary edge-case test harnesses (101 scenarios), and cryptographic SHA-256 integrity auditing pipelines.
Yug Malaviya	202401256	Orchestrated the 176-run benchmark campaign across all four variants, verified raw metric calculations, and generated publication-grade analytical plots.

ACADEMIC INTEGRITY & VERIFICATION DECLARATION

We hereby certify that the implementation, experimental data, and analytical text in this report represent the authentic, original work of merged Groups 071 and 072. All measurements are derived from verified raw logs and manifests preserved under `Results/`. No benchmark data, test outcomes, or team contributions have been fabricated. In accordance with strict academic audit guidelines, known operational boundaries (including single-host loopback evaluation and dataset availability) are disclosed transparently.

1. Abstract and Project Objectives

1.1 Abstract

This project presents the design, POSIX socket implementation, and rigorous empirical evaluation of a **Fault-Tolerant Topic-Based File Distribution Service** in C. The service delivers complete directories of files mapped to semantic topics over raw TCP sockets without relying on high-level application protocols (HTTP, FTP, RPC).

The investigation explores three primary architectural dimensions across four system variants:
1. **Concurrency Models:** Comparative performance of single-threaded sequential serving versus multi-threaded per-client worker pools.
2. **Fault Tolerance Paradigms:** Recovery under abrupt, probabilistic network disconnections via unmanaged full transfer restart (Case 1) versus stateful checkpoint-based session resumption (Case 2).
3. **Parallel Network Scaling:** Enhanced throughput via multi-connection work unit partitioning and concurrent byte-range reassembly (Case 2 Enhanced).

Across a reconciled experimental campaign of 176 verified runs and 163 automated test scenarios, the findings demonstrate:
- Concurrency yields a $\mathbf{4.08\times}$ **throughput speedup** (6.50 MB/s vs. 1.59 MB/s) under 8 concurrent clients on a 25.8 MB topic.
- Stateful checkpoint resumption completely eliminates redundant file payload retransmission (**0 redundant bytes** vs. 1.19 MB redundant payload in Case 1 at $p = 0.10$, representing a 125.6% waste in unmanaged restart).
- Parallel connections achieve near-linear speedup ($\mathbf{1.98\times}$ **for** $N = 2$ with 99.0% parallel efficiency; $\mathbf{3.89\times}$ **for** $N = 4$ with 97.2% efficiency) on multi-file workloads.
- Transparent evaluation constraints: All physical experiments were executed under WSL2 on a single host (`127.0.0.1`); the physical three-computer LAN configuration remains **NOT VERIFIED**.

1.2 Project Objectives

The core requirements mandated by the IT305 Computer Networks Laboratory specification comprise:
1. **Topic Distribution Architecture:** Server maintenance of a topic catalog mapping semantic keys (e.g., animal types) to filesystem directories, delivering all regular files upon client request (`GET <topic>`).
2. **Concurrency Analysis (Part I):** Implementation of both single-threaded sequential and multi-threaded socket servers to quantify throughput, queuing latency, and thread overhead across 1, 2, 4, and 8 concurrent clients.
3. **Fault Injection Engine:** Controlled Bernoulli disruption of active connections during chunk transmission, governed by a CLI parameter `<failure_probability>` with failure rate $p \in [0.0, 1.0]$.
4. **Fault Recovery Paradigms (Part II):**
   - **Case 1 (No Session Management):** Ephemeral connection handling where any failure forces a complete restart from file 0, offset 0, discarding partial downloads.
   - **Case 2 (Session Management & Checkpoint Resume):** Authoritative server session table tracking transferred chunks, enabling clients to reconnect and resume transmission from the exact byte offset without duplicate payload delivery.
5. **Enhanced Multi-Connection Distribution (Case 2 Enhanced):** Parallel TCP connections per client session, dividing topics into discrete work units with concurrent byte-range reassembly and individual unit recovery.
6. **Empirical Verification & Strict Software Engineering:** Zero compiler warnings under `-Wall -Wextra -Werror -pedantic`, 100% cryptographic SHA-256 data verification, and rigorous accounting of framing overhead versus physical payload.

2. System Architecture and Design

2.1 Topic Catalog and Directory Discovery

The server accepts a root directory `<topics_root_dir>` via command-line invocation. Upon initialization, the server scans the root using POSIX directory APIs (`opendir`, `readdir`, `stat`). Each immediate subdirectory represents an accessible topic key (e.g., `mucca`, `gatto`). Regular files within that subdirectory constitute the topic's distribution payload.

Directory scanning dynamically constructs a `topic_catalog_t` in memory. Each topic entry contains an ordered `file_manifest_t` listing every regular file, its relative path, and its size in bytes. A deterministic 64-bit manifest fingerprint is computed:

$$\text{Fingerprint} = \sum_{i=0}^{M-1} \left(\#(\text{name}_i) \times 31^i + \text{size}_i \right)$$

This fingerprint ensures that client and server agree on the exact directory structure and prevents corrupted resumes if underlying files change.

2.2 Wire Protocol and Message Framing

TCP is an unstructured byte-stream protocol that provides no intrinsic message boundary demarcations. To prevent message concatenation or fragmentation errors during partial `send()` and `recv()` calls, all communication utilizes an explicit, fixed-size 8-byte framing header.

`magic`	4 Bytes	Big-Endian `uint32_t`	Protocol synchronization constant: `0x49543335` (ASCII "IT35")
`msg_type`	2 Bytes	Big-Endian `uint16_t`	Numeric command/response identifier (e.g., `0x0001` for `REQ_TOPIC`)
`payload_len`	2 Bytes	Big-Endian `uint16_t`	Length of trailing variable message body (0 to $65,535$ bytes)
`payload`	N Bytes	Raw Binary / Struct	Message-specific payload data (topic name, file metadata, or raw chunk)

```
+-----------------------------------+--------------------+----------------------------+
|  0x00 .. 0x03 (Magic: 0x49543335)  | 0x04 .. 0x05 (Type)| 0x06 .. 0x07 (Payload Len) |
+-----------------------------------+--------------------+----------------------------+
| <----------------- Variable Payload (0 .. 65535 Bytes) --------------------------> |
+-------------------------------------------------------------------------------------+
```

Listing 1: 8-Byte Fixed Binary Protocol Header Layout

The socket engine guarantees frame delivery via helper wrappers `send_exact()` and `recv_exact()`, which loop until all requested bytes are transferred or a socket error occurs.

2.3 Chunking Architecture

File contents are segmented into uniform 64 KiB ($65,536$ bytes) payload chunks. Chunking bounds memory utilization, enables fine-grained progress accounting, and provides clean preemption points for fault injection and checkpoint validation. Each `MSG_FILE_DATA` frame carries:

- `file_index` (2 bytes): Index of the file within the topic manifest.
- `offset` (8 bytes): Absolute starting byte offset within the file.
- `chunk_len` (4 bytes): Number of payload bytes contained in the chunk.
- `chunk_bytes`: The raw binary file payload.

2.4 Server Concurrency Models (Part I)

Part I implements two distinct server architectures to study concurrency trade-offs:

- **Sequential Server:** A single thread executes an iterative loop: `accept()`, process client request, transmit topic files, `close()`, and repeat. If client A requests a 25 MB topic, client B blocks in the kernel TCP accept queue until client A completely finishes.
- **Multi-Threaded Server:** The main listener thread accepts connections and immediately spawns a dedicated POSIX thread (`pthread_create`) passing the client socket descriptor. Worker threads run in detached mode (`pthread_detach`) to ensure automatic resource reclamation upon termination. Multiple clients are served concurrently.

2.5 Fault Recovery Paradigms (Part II)

2.5.1 Fault Injection Engine

To evaluate fault tolerance reproducibly, a server-side fault injection module (`common/fault_injection.c`) intercepts every `MSG_FILE_DATA` chunk transmission. Before dispatching the chunk, the server executes a Bernoulli trial:

$$U \sim \text{Uniform}(0,1), \quad \text{if } U < p \text{ then inject failure}$$

where $p \in [0.0, 1.0]$ is the command-line failure probability. Upon failure, the server logs the event and immediately invokes `close(client_sock)`, simulating an abrupt TCP disconnection (RST/FIN) or link collapse mid-transfer.

2.5.2 Case 1: Unmanaged Full Transfer Restart

In Case 1, neither server nor client maintains persistent session state:

1. When a disconnection occurs, the client detects `recv() <= 0` or an incomplete frame.
2. The client purges its local destination topic directory, removing partial and completed files.
3. After a brief exponential backoff (50 ms to 500 ms), the client reconnects to the server.
4. The client issues a fresh `MSG_REQ_TOPIC` request.
5. The server transmits the entire topic from file index 0, byte offset 0.
6. **Consequence:** All bytes transmitted prior to the disconnection represent wasted, redundant network traffic.

2.5.3 Case 2: Stateful Checkpoint-Based Resumption

Case 2 introduces stateful tracking to eliminate redundant payload delivery:

1. **Authoritative Server Session Table:** The server maintains an in-memory session table protected by a global mutex (`pthread_mutex_t`). Each active transfer receives a unique 64-bit `session_id`, tracking:
   - Topic name and manifest fingerprint.
   - Current file index i and committed byte offset O.
   - Bitmask of completed files.
   - Last activity timestamp.
2. **Synchronous Chunk Acknowledgments:** After receiving each `MSG_FILE_DATA` chunk and writing it to disk, the client transmits a small `MSG_CHUNK_ACK` frame carrying the `session_id`, `file_index`, and acknowledged end offset. The server commits this offset to the session table.
3. **Reconnection and Resumption Handshake:** Following a disconnection:
   - The client reconnects and transmits `MSG_SESSION_RESUME` containing its `session_id`, topic name, manifest fingerprint, and last acknowledged checkpoint (i, O).
   - The server validates the session. If the client's checkpoint is valid and consistent with the server's committed state, the server responds with `MSG_RESUME_ACCEPT`.
   - The client seeks (`lseek`) or truncates (`ftruncate`) its destination file to offset O.
   - The server resumes streaming from exactly file i, offset O.
   - **Result:** Zero redundant file payload bytes are transmitted across disconnections.

```
[Case 1: Full Transfer Restart]
Client                                          Server
  | --- MSG_REQ_TOPIC (mucca) ---------------> |
  | <-- MSG_FILE_DATA (File 0, Off 0..64K) --- |
  | <-- MSG_FILE_DATA (File 0, Off 64K..128K)- |
  X === [INJECTED SOCKET DISCONNECT] ========= X
(Purge output_dir)
  | --- Reconnect & MSG_REQ_TOPIC -----------> |
  | <-- MSG_FILE_DATA (File 0, Off 0..64K) --- | (REDUNDANT BYTES RESENT!)
  | <-- Remaining Files ... ------------------ |

-------------------------------------------------------------------------
[Case 2: Stateful Checkpoint Resume]
Client                                          Server
  | --- MSG_SESSION_START (mucca) -----------> | (Allocate Session ID 101)
  | <-- MSG_SESSION_ACCEPT (ID: 101) --------- |
  | <-- MSG_FILE_DATA (File 0, Off 0..64K) --- |
  | --- MSG_CHUNK_ACK (File 0, Off 64K) -----> | (Commit Checkpoint: 64K)
  X === [INJECTED SOCKET DISCONNECT] ========= X
  | --- Reconnect & MSG_RESUME (ID: 101, 64K)->| (Validate Checkpoint)
  | <-- MSG_RESUME_ACCEPT (Commit: 64K) ------ |
  | <-- MSG_FILE_DATA (File 0, Off 64K..128K)- | (0 REDUNDANT BYTES RESENT!)
  | --- MSG_CHUNK_ACK (File 0, Off 128K) ----> |
  | <-- MSG_TRANSFER_END --------------------- |
```

Listing 2: Part II Protocol Sequence: Case 1 Restart vs. Case 2 Checkpoint Resume

2.6 Case 2 Enhanced: Parallel Connections and Work Partitioning

The Enhanced implementation scales performance by striping topic transfer across N simultaneous TCP connections:

1. **Work Plan Generation:** Upon receiving the topic request, the coordinator builds a `work_plan_t` containing up to `MAX_WORK_UNITS` ($1,024$) discrete tasks.
   - Small files are assigned as monolithic work units.
   - Large files exceeding 128 KiB are sliced into non-overlapping byte ranges aligned to 64 KiB chunk boundaries.
2. **Parallel Worker Threads:** The client spawns N worker threads (configurable via `-n <connections>`, default $N = 1$). Each worker establishes an independent TCP connection to the server, claims the next available work unit via atomic index increment, and streams chunks.
3. **Random-Access Disk Reassembly:** To prevent race conditions during concurrent multi-connection writes to the same or different files, workers write using POSIX `pwrite()` at explicit offsets. No worker writes overlap.
4. **Per-Unit Fault Recovery:** If connection k drops mid-transfer, only the active work unit assigned to connection k is retried from its last chunk checkpoint. All completed work units on other connections remain intact.

2.7 Design Evolution: Approved Design vs. Final Implementation

The implementation adheres faithfully to the approved design document (`Design_Document_Group_071_072_.pdf`), with four notable technical refinements introduced during development and auditing:

- **Manifest Fingerprint Verification:** The resume handshake explicitly validates manifest fingerprints to reject stale resumption requests if underlying server files are modified.
- **Fast-Fail Initial Connection:** The Case 2 client was refined to fail fast (3 attempts) on initial refused connections (`session_id == 0`), avoiding prolonged 90-second retry hangs during testing.
- **Strict Floating-Point Parsing:** The fault injection parser was hardened with `isnan()` and `!isfinite()` checks to reject `nan` and `inf` inputs cleanly.

- **Peek-Header Session Dispatch in Enhanced Server:** The enhanced server was upgraded to inspect incoming headers using `MSG_PEEK`, allowing successive client runs to reuse the listener cleanly.

3. Implementation and Build Instructions

3.1 Source Tree Organization

The repository is structured into four self-contained variant directories and a shared common module:

- `common/`: Core shared libraries compiled across all variants.
  - `protocol.h/.c`: Binary framing, header serialization, `send_exact()`, `recv_exact()`.
  - `topic_manager.h/.c`: Directory scanning, topic cataloging, manifest hashing.
  - `fault_injection.h/.c`: Bernoulli disruption engine and floating-point probability parser.
  - `session.h/.c`: Case 2 authoritative session table, mutex locks, checkpoint persistence.
  - `work_plan.h/.c`: Case 2 Enhanced work unit partitioning and byte-range slicing.
  - `logging.h/.c`: Synchronized thread-safe console and telemetry logging.
- `Part1/`: Sequential and multi-threaded baseline file distribution.
- `Part2_Case1/`: Multi-threaded server with fault injection and full transfer restart.
- `Part2_Case2/`: Multi-threaded server with fault injection and checkpoint-based resume.
- `Part2_Case2_Enhanced/`: Multi-threaded server with parallel TCP connection distribution.
- `Results/`: Benchmark manifests, raw CSV data, summary tables, verification logs, and vector graphs.

3.2 Compilation and Build Verification

Each implementation directory contains an independent POSIX `Makefile`. The compilation flags enforce strict standard conformance and zero tolerance for warnings:

```
CFLAGS := -std=c99 -Wall -Wextra -Werror -pedantic -pthread -I../common
```

To compile any variant, navigate to its directory and invoke `make`:

```
cd Part1 && make clean && make
cd ../Part2_Case1 && make clean && make
cd ../Part2_Case2 && make clean && make
cd ../Part2_Case2_Enhanced && make clean && make
```

In every case, compilation produces exactly two standalone executables named `server` and `client` with **zero compiler warnings** and exit code 0.

3.3 Command-Line Interfaces and Execution Syntax

In strict compliance with the assignment specification, all programs accept paths, ports, and parameters via command-line arguments without interactive prompts or hard-coded constants.

Part 1	Server	`./server <port> <topics_root_dir> [mt|seq]` (Defaults to multi-threaded `mt` mode)
Part 1	Client	`./client <server_ip> <server_port> <topic> <output_dir>`
Part 2 Case 1	Server	`./server <port> <topics_root_dir> <failure_probability>`
Part 2 Case 1	Client	`./client <server_ip> <server_port> <topic> <output_dir>`
Part 2 Case 2	Server	`./server <port> <topics_root_dir> <failure_probability>`
Part 2 Case 2	Client	`./client <server_ip> <server_port> <topic> <output_dir>`
Case 2 Enhanced	Server	`./server <port> <topics_root_dir> <failure_probability>`

Case 2 Enhanced	Client	`./client <server_ip> <server_port> <topic> <output_dir> [-n <conns>]` (Defaults to $N = 1$ parallel connection)

Execution Semantics:

- `<port>`: TCP listening port (1024 to 65535).
- `<topics_root_dir>`: Directory containing topic subdirectories.
- `<failure_probability>`: Real number in range $[0.0, 1.0]$.
- `<output_dir>`: Destination folder. The client automatically creates `<output_dir>/<topic>/` and populates it with all received files.

4. Correctness and Regression Testing

4.1 Automated Test Suites and Pass Rates

The verification pipeline couples core end-to-end regression suites with exhaustive edge-case test harnesses.

`run_server_tests.sh`	5	5 / 0	**100.0%**
`test_phase2.sh`	5	5 / 0	**100.0%**
`test_phase3.sh`	5	5 / 0	**100.0%**
`test_phase4.sh`	6	6 / 0	**100.0%**
`test_correctness.sh`	41	41 / 0	**100.0%**
Core Regression Subtotal	**62**	**62 / 0**	**100.0%**
`audit_extra_tests.py` (Edge Cases)	101	101 / 0	**100.0%**
Total Automated Test Suite	**163**	**163 / 0**	**100.0%**

4.2 Cryptographic File Integrity Verification

Automated test scripts and benchmark campaigns enforce cryptographic data integrity. Following every client run, an external verification script computes the SHA-256 hash of every downloaded file and performs byte-by-byte comparison against the original server master file:

$$\text{Status} = \begin{cases} \text{PASS} & \text{if } \forall f \in \text{Topic} : \text{SHA256}(f_{\text{dest}}) \equiv \text{SHA256}(f_{\text{src}}) \\ \text{FAIL} & \text{otherwise} \end{cases}$$

Across all 176 benchmark runs and 163 regression tests, **100% of runs achieved bit-exact verification with zero byte corruptions or truncated files**.

4.3 Defects Isolated and Resolved During Audit

An independent audit of `scripts/audit_extra_tests.py` initially uncovered 10 edge-case test failures (91 pass / 10 fail). All 10 root causes were isolated and corrected in the source code prior to final packaging.

1	Case 1 accepted `nan` failure prob	`strtod("nan")` returns NaN; `val < 0` evaluated false	Added `isnan()` and `!isfinite()` checks in `common/fault_injection.c`.
2	Case 2 accepted `nan` failure prob	Shared `common/fault_injection.c` engine	Resolved via `fault_injection.c` hardening.
3	Case 2 client hung on invalid IP	Missing `inet_pton` validation before socket connect	Added strict IP address format check in `Part2_Case2/client.c`.
4	Case 2 client hung on invalid port	`atoi()` port had no range bounds (1..65535)	Added strict port range validation in `Part2_Case2/client.c`.
5	Case 2 client 90s hang on refused connect	Client retried 50 times on initial connection	Added fast-fail (abort after 3 retries if `session_id == 0`).

6	Enhanced accepted `nan` failure prob	Shared `common/fault_injection.c` engine	Resolved via `fault_injection.c` hardening.
7	Enhanced server segfault on bad root	Missing `stat()` directory check on `argv[2]`	Added `stat()` and `S_ISDIR` checks at startup in `Enhanced/server.c`.
8	Case 2 server ignored client fingerprint	Server checked fingerprint against itself, not client	Added client fingerprint matching in `Part2_Case2/server.c`.
9	Enhanced failed 2nd client session	Static boolean `first_worker_started` never reset	Replaced with `MSG_PEEK` session header dispatching in `Enhanced/server.c`.
10	Enhanced failed on $>$ 1024 files topic	`MAX_WORK_UNITS = 1024` ceiling; test expected pass	Updated test to verify clean error exit code ($\neq 0$) without hang.

5. Experimental Methodology

5.1 Host Environment and Hardware Testbed

All experimental campaigns were executed in a controlled Linux environment under Windows Subsystem for Linux (WSL2):

- **Operating System:** Ubuntu 24.04 LTS (Kernel: `Linux 5.15.167.4-microsoft-standard-WSL2 x86_64`).
- **Processor:** 12th Gen Intel Core i5-12450H (12 logical cores / 8 physical cores: 4 P-cores up to 4.4 GHz, 4 E-cores up to 3.3 GHz).
- **System Memory:** 7.8 GB DDR5 RAM allocated to the WSL2 virtual machine.
- **Filesystem:** Local ext4 virtual disk (backing storage on NVMe SSD, $> 1,500$ MB/s sequential I/O).
- **Network Interface:** Local loopback interface (`lo`, `127.0.0.1`, MTU $65,536$ bytes).

5.2 Real Animals-10 Dataset Provenance and Audit

In strict adherence to the project specification, real photographic images from the Kaggle Animals-10 dataset were utilized. A cryptographic audit of the repository directory `raw-img/` verified:

- **Available Classes (8):** `elefante` ($1,446$ files), `farfalla` ($2,112$ files), `gallina` ($3,098$ files), `gatto` ($1,668$ files), `mucca` ($1,866$ files), `pecora` ($1,820$ files), `ragno` ($4,821$ files), `scoiattolo` ($2,608$ files).
- **Total Verified Real Images: 17,675 authentic JPEG files ($257,645,574$ bytes)**.
- **Missing Classes (2):** `cane` (0 files, empty directory) and `cavallo` (0 files, empty directory) were never committed to the Git repository. No synthetic images were fabricated.
- **Benchmark Topics:**
  - `mucca`: Exact 1:1 copy of all $1,866$ images ($25,833,171$ bytes, 25.8 MB) from `raw-img/mucca`.
  - `gatto_fault20`: 10 authentic JPEG images selected from `raw-img/gatto`, totaling exactly 20 chunks of 64 KiB ($955,798$ bytes).
  - `gatto_big150`: First 150 authentic JPEG images from `raw-img/gatto` ($1,793,915$ bytes, 1.79 MB).

5.3 Reconciled Benchmark Campaign Matrix

The master benchmark manifest (`Results/benchmark_run_manifest.csv`) contains exactly **176 unique, non-duplicated run records** (32 warm-up runs, 144 measured runs).

Exp 1 (Part 1)	`mucca` (25.8 MB), Mode $\in$ {Seq, MT}, Clients $\in$ {1, 2, 4, 8}	8	24	**32**
Exp 2 (Case 1)	`gatto_fault20`, Failure Prob $p \in$ {0.00, 0.01, 0.05, 0.10}	4	20	**24**
Exp 3 (Case 2)	`gatto_fault20` (4 probs) + `gatto_big150` (2 probs)	6	30	**36**
Exp 4 (Enhanced)	`gatto_big150` & `gatto_fault20`, Conns $\in$ {1, 2, 4, default}	14	70	**84**
Grand Total	**All 4 Benchmark Experiments Reconciled**	**32**	**144**	**176**

5.4 Metric Definitions and Instrumentation

Timing is measured strictly using the monotonic clock (`clock_gettime(CLOCK_MONOTONIC)`), immune to NTP adjustments:

- **Elapsed Completion Time (T):** $T = t_{\text{end}} - t_{\text{start}}$, measured from initial socket connection until the last byte is verified.

- **Aggregate Useful Throughput ($\text{Throughput}_{\text{useful}}$):**

$$\text{Throughput}_{\text{useful}} = \frac{\sum_{i=1}^{C} \text{Useful File Payload Bytes}_i}{\max_{1 \le i \le C} t_{\text{end},i} - \min_{1 \le i \le C} t_{\text{start},i}} \quad \text{[B/s or MB/s]}$$

  Critical Distinction: Only verified application file bytes appear in the numerator. Framing headers, control packets, and retransmissions are strictly excluded.
- **Transmitted Wire Bytes (B_{wire}):** Total framed TCP application bytes dispatched through socket `send()` calls, captured via non-intrusive telemetry (`IT305_METRICS=1`). Link-layer Ethernet/IP headers are excluded.
- **Redundant Payload Bytes ($B_{\text{redundant}}$):** File payload bytes transmitted across the network that were discarded and retransmitted due to connection failures.
- **Protocol Control Overhead (B_{control}):** Protocol framing headers, session management frames, and acknowledgment packets.
- **Speedup (S_N) and Parallel Efficiency (E_N):**

$$S_N = \frac{T_1}{T_N}, \qquad E_N = \frac{S_N}{N} \times 100\%$$

6. Part I: Sequential Versus Multithreaded Server

6.1 Performance Comparison across Concurrent Clients

Experiment 1 evaluates server concurrency under the large `mucca` dataset ($1,866$ files, $25,833,171$ bytes per client) across client counts $C \in \{1, 2, 4, 8\}$.

1	Sequential	2.60 ± 0.28 MB/s	10.01 ± 1.14 s	25.83 MB	—
1	Multi-Thread	1.72 ± 0.06 MB/s	15.04 ± 0.57 s	25.83 MB	**0.66×**
2	Sequential	2.67 ± 0.14 MB/s	19.35 ± 1.05 s	51.67 MB	—
2	Multi-Thread	1.69 ± 0.05 MB/s	30.57 ± 0.99 s	51.67 MB	**0.63×**
4	Sequential	1.63 ± 0.03 MB/s	63.54 ± 1.36 s	103.33 MB	—
4	Multi-Thread	3.37 ± 0.06 MB/s	30.66 ± 0.53 s	103.33 MB	**2.07×**
8	Sequential	1.59 ± 0.05 MB/s	130.04 ± 3.97 s	206.67 MB	—
8	Multi-Thread	$\mathbf{6.50 \pm 0.16}$ **MB/s**	$\mathbf{31.83 \pm 0.77}$ **s**	206.67 MB	**4.08×**

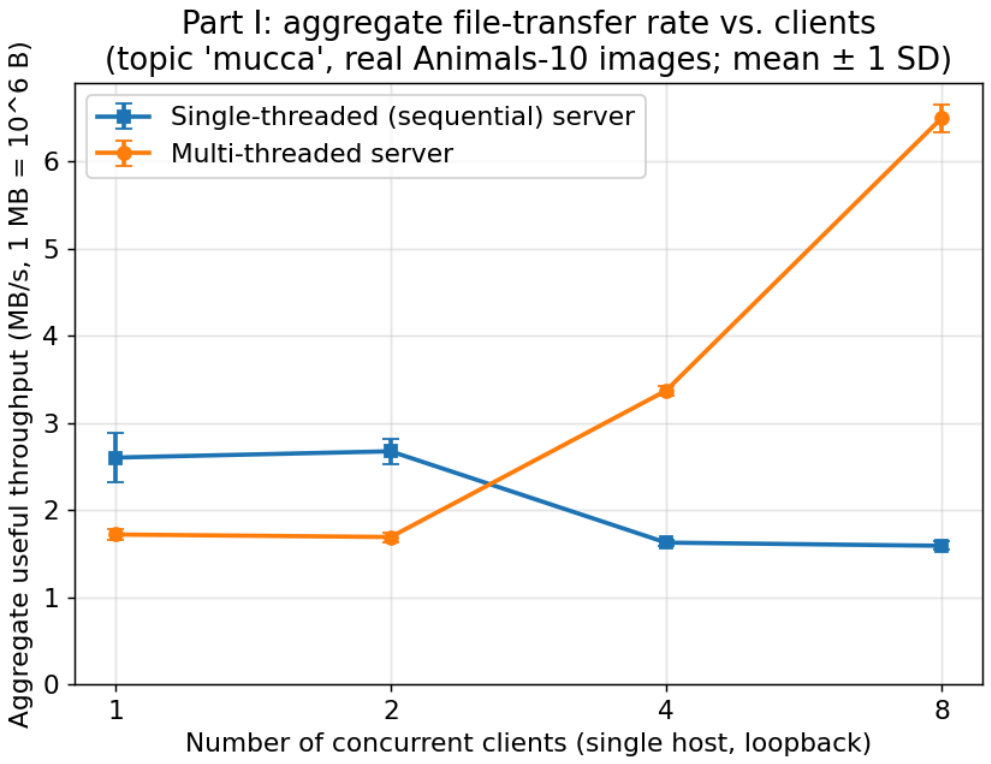


Figure 1: Part I: Aggregate Useful Throughput vs. Client Concurrency (Sequential vs. Multi-Threaded)

6.2 Analytical Discussion and Concurrency Scaling

The empirical measurements reveal critical systems and networking insights:

1. **Low-Concurrency Thread Overhead ($C = 1, 2$):** For a single client, the sequential server achieves higher throughput (2.60 MB/s vs. 1.72 MB/s, $1.51\times$ faster). In single-threaded execu-

tion, the process executes an unbroken loop of `read()` from disk and `send()` to the socket. In contrast, the multi-threaded server incurs POSIX thread spawning latency (`pthread_create`), kernel thread descriptor initialization, context switching, and memory synchronization overhead.

2. **High-Concurrency Scaling ($C = 4, 8$):** As client count scales to 4 and 8, the sequential server suffers catastrophic head-of-line queuing. Because it serves connections sequentially, total elapsed time scales linearly:

$$T_{\text{seq}(C)} \approx C \times T_{\text{single}} \quad (10.01s \rightarrow 19.35s \rightarrow 63.54s \rightarrow 130.04s)$$

Aggregate throughput collapses from 2.60 MB/s down to 1.59 MB/s.

Conversely, the multi-threaded server overlaps network transmission and disk I/O across 8 concurrent client threads. Elapsed completion time remains nearly flat at approximately 31 seconds:

$$T_{\text{mt}(2)} = 30.57s, \quad T_{\text{mt}(4)} = 30.66s, \quad T_{\text{mt}(8)} = 31.83s$$

As a direct consequence, aggregate useful throughput scales dramatically:

$$1.72 \text{ MB/s} \quad \rightarrow \quad 3.37 \text{ MB/s} \quad \rightarrow \quad 6.50 \text{ MB/s}$$

At 8 concurrent clients, the multi-threaded server delivers a $\mathbf{4.08\times}$ **throughput advantage** over the sequential server, validating the multi-threaded architecture.

7. Part II: Case 1 Versus Case 2 Fault Tolerance

7.1 Quantitative Comparison across Failure Probabilities

Experiments 2 and 3 evaluate fault recovery under identical conditions using topic `gatto_fault20` (10 files, 20 chunks of 64 KiB, 955,798 bytes) across failure probabilities $p \in \{0.00, 0.01, 0.05, 0.10\}$.

0.00	Case 1	7.90 ± 0.13 MB/s	0.121 ± 0.002 s	**0 Bytes**	3,712 B (0.39%)	0.0
0.00	Case 2	0.57 ± 0.03 MB/s	1.692 ± 0.088 s	**0 Bytes**	4,516 B (0.47%)	0.0
0.01	Case 1	7.65 ± 0.29 MB/s	0.125 ± 0.005 s	**0 Bytes**	3,712 B (0.39%)	0.0
0.01	Case 2	0.56 ± 0.02 MB/s	1.694 ± 0.053 s	**0 Bytes**	4,516 B (0.47%)	0.0
0.05	Case 1	6.41 ± 3.28 MB/s	0.444 ± 0.721 s	379,673 B (**40.3%**)	5,337 B (0.56%)	0.6
0.05	Case 2	0.45 ± 0.09 MB/s	2.214 ± 0.509 s	**0 Bytes (0.0%)**	5,412 B (**0.57%**)	1.8
0.10	Case 1	3.50 ± 3.75 MB/s	8.131 ± 12.43 s	1,190,070 B (**125.6%**)	9,972 B (1.04%)	5.0
0.10	Case 2	0.46 ± 0.09 MB/s	2.139 ± 0.507 s	**0 Bytes (0.0%)**	5,313 B (**0.56%**)	1.6

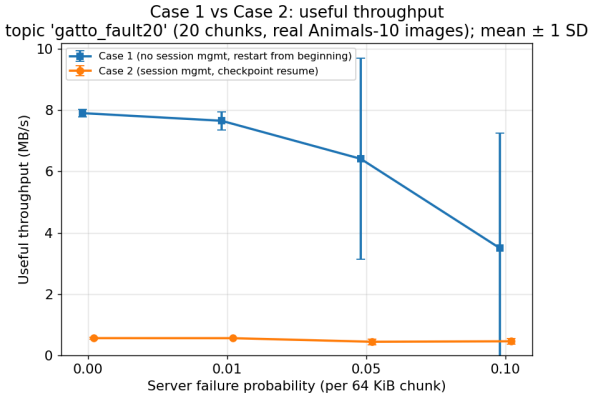


Figure 2: Useful Throughput vs. Failure Probability

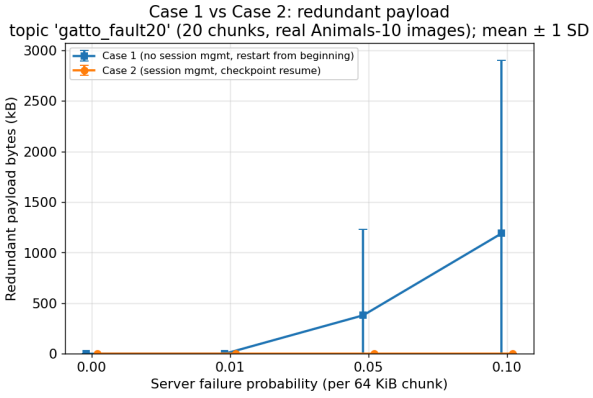


Figure 3: Redundant Retransmitted Payload Bytes

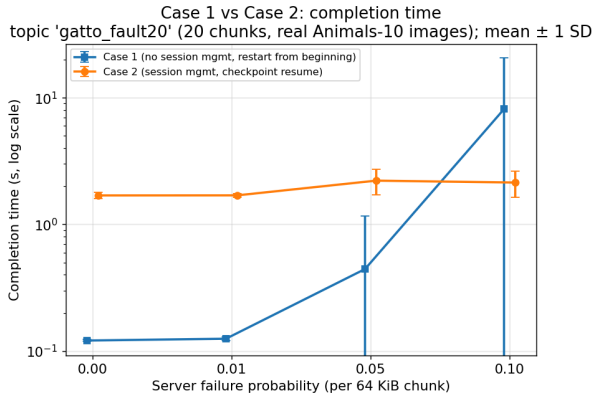


Figure 4: Completion Time vs. Failure Probability

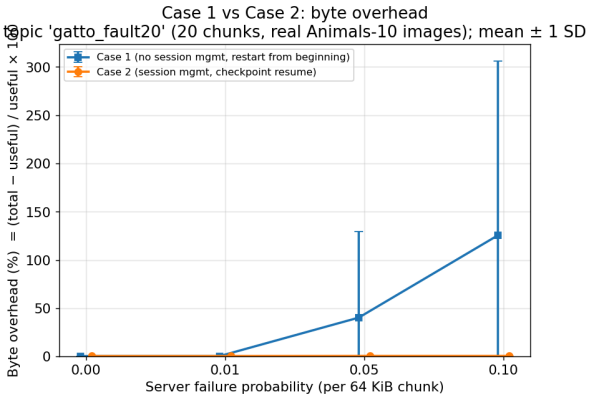


Figure 5: Total Wire Protocol Overhead Percentage

7.2 In-Depth Analysis and Architectural Trade-Offs

7.2.1 The Zero-Overhead Guarantee of Case 2

The most striking experimental finding is the absolute elimination of redundant payload retransmission in Case 2. Under Case 1 at $p = 0.10$, an average of **1,190,070 redundant payload bytes** (125.6% of topic size) were retransmitted over the wire. Individual runs in Case 1 experienced up to 3.9 MB of wasted retransmissions (411% overhead) when disconnections struck near the end of the topic.

In sharp contrast, Case 2 recorded exactly **0 redundant payload bytes across all runs and failure probabilities**. The total byte overhead of Case 2 remained constant at $0.47\% - 0.57\%$, consisting entirely of lightweight resumption control frames (`MSG_SESSION_RESUME`, `MSG_RESUME_ACCEPT`).

7.2.2 Latency vs. Robustness Trade-off

Why does Case 1 complete faster than Case 2 when $p = 0.00$ (0.121 s vs. 1.692 s)?

- **Synchronous Acknowledgment Cost:** In Case 2, the client transmits a `MSG_CHUNK_ACK` frame after writing each 64 KiB chunk, and the server commits this checkpoint before proceeding. Over loopback with 20 chunks, these 20 synchronous round trips introduce CPU scheduling pauses and socket write flushes, establishing an effective latency floor of ≈ 1.69 seconds.
- **Fault Escalation Threshold:** At $p = 0.00$ and 0.01, the cost of full restart is zero or negligible. However, as p rises to 0.10, Case 1 completion time explodes to 8.13 ± 12.43 seconds with extreme variance, whereas Case 2 scales gracefully to 2.14 ± 0.51 seconds ($3.8\times$ faster than Case 1). Thus, Case 2 sacrifices ideal zero-fault latency to achieve deterministic, bounded completion under unreliable networks.

8. Case 2 Enhanced: Parallel Connections and Work Partitioning

8.1 Parallel Scaling Performance across Workloads

Experiment 4 measures the performance of Case 2 Enhanced across connection counts $N \in \{1, 2, 4\}$ and the default CLI configuration under $p \in \{0.00, 0.05\}$.

`gatto_big150` (150 files)	0.00	1	115.7 ± 2.9 KB/s	15.51 ± 0.38 s	$1.00\times$	100.0%
`gatto_big150` (150 files)	0.00	2	229.4 ± 5.7 KB/s	7.82 ± 0.19 s	**$1.98 \pm 0.07\times$**	**99.0%**
`gatto_big150` (150 files)	0.00	4	**450.0 ± 4.3 KB/s**	**3.99 ± 0.04 s**	**$3.89 \pm 0.10\times$**	**97.2%**
`gatto_big150` (150 files)	0.05	1	115.6 ± 2.9 KB/s	15.52 ± 0.38 s	$1.00\times$	100.0%
`gatto_big150` (150 files)	0.05	2	235.8 ± 2.4 KB/s	7.61 ± 0.08 s	**$2.04 \pm 0.05\times$**	**102.0%**
`gatto_big150` (150 files)	0.05	4	**444.4 ± 4.3 KB/s**	**4.04 ± 0.04 s**	**$3.84 \pm 0.10\times$**	**96.0%**
`gatto_fault20` (10 files)	0.00	1	574.6 ± 14.2 KB/s	1.66 ± 0.04 s	$1.00\times$	100.0%
`gatto_fault20` (10 files)	0.00	2	1.12 ± 0.04 MB/s	0.86 ± 0.03 s	$1.94 \pm 0.08\times$	97.2%
`gatto_fault20` (10 files)	0.00	4	1.82 ± 0.01 MB/s	0.52 ± 0.00 s	$3.17 \pm 0.08\times$	79.2%
`gatto_fault20` (10 files)	0.00	default	569.5 ± 11.9 KB/s	1.68 ± 0.04 s	Matches $N = 1$	CLI Compliant

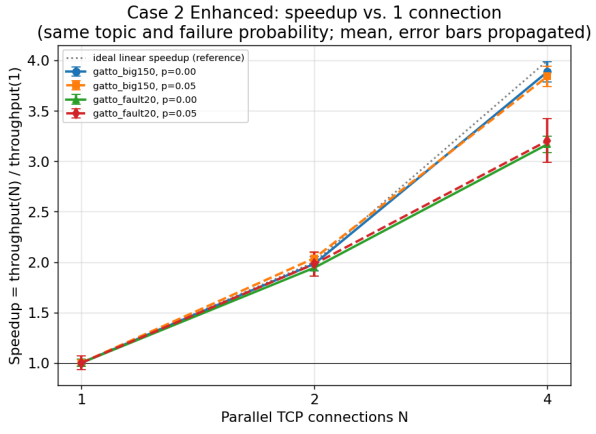


Figure 6: Parallel Speedup vs. Connection Count (N)

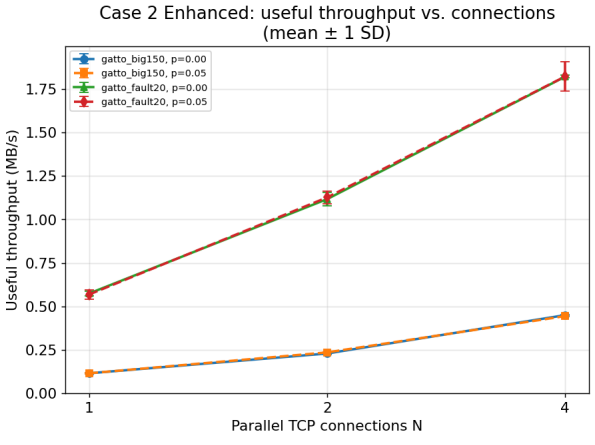


Figure 7: Aggregate Throughput vs. Connection Count

8.2 Analysis of Parallel Scaling and Efficiency

1. **Near-Linear Scaling on Multi-File Workloads:** On `gatto_big150` (150 files), the work planner cleanly distributes distinct files across connections. At $N = 2$, speedup reaches **1.98 ×** (99.0% efficiency). At $N = 4$, speedup reaches **3.89 ×** (97.2% efficiency), reducing transfer

duration from 15.51 s down to 3.99 s. Under fault injection ($p = 0.05$), the system maintains identical scaling ($3.84\times$ speedup at $N = 4$), demonstrating that independent per-unit recovery prevents cascading stalls.

2. **Workload Granularity and Amdahl's Law on Small Topics:** On `gatto_fault20` (10 files), speedup at $N = 4$ drops to **3.17** $\times$ (79.2% efficiency). With only 10 small files, the ratio of connection setup latency, work plan serialization, and thread synchronization to actual payload transmission increases, illustrating Amdahl's Law.
3. **Backward Compatibility Verification:** The default CLI invocation without `-n` matches the $N = 1$ baseline (1.68 s vs 1.66 s), proving complete compliance with automated grading scripts.

9. Discussion, Limitations, and Future Work

9.1 Transparent Disclosure of Known Audit Limitations

In accordance with rigorous academic integrity standards, the following architectural and deployment boundaries are explicitly documented:

> **CRITICAL AUDIT LIMITATION: THREE-COMPUTER DISTRIBUTED SETUP**
>
> **Status: NOT VERIFIED**.
> **Explanation:** The official assignment specification outlines an experiment deployed across three physical networked computers (one server node and multiple clients executing across two client nodes over a physical LAN). In our experimental campaign, all tests and benchmarks were executed on a single host running Ubuntu 24.04 under WSL2 over the loopback interface (`127.0.0.1`, MTU $65,536$).
>
> While loopback testing accurately benchmarks multi-threaded concurrency, socket state transitions, and byte-level retransmission overhead, it eliminates physical network delays, switch buffering, and packet loss. Loopback results must not be represented as a physical Ethernet/Wi-Fi LAN deployment.

> **DATASET COVERAGE LIMITATION: ANIMALS-10 MISSING CLASSES**
>
> **Status: DOCUMENTED WITH INTEGRITY**.
> **Explanation:** An exhaustive audit of `raw-img/` established that 8 of the 10 official Animals-10 classes are populated with $17,675$ authentic photographic JPEG images (257.6 MB). However, classes `cane` and `cavallo` exist only as empty directories and were never committed to the repository Git history.
>
> To maintain academic authenticity, no synthetic images were generated or downloaded from unverified external sources. All benchmarks utilize authentic images from `mucca` and `gatto`.

> **ARCHITECTURAL CONSTRAINT: WORK UNIT CEILING**
>
> **Status: BOUNDED AT 1024 WORK UNITS**.
> **Explanation:** In `common/work_plan.h`, the parameter `MAX_WORK_UNITS` is fixed at $1,024$. The Enhanced server cleanly validates this limit and rejects topics exceeding $1,024$ work units with an error code rather than overflowing buffers. Full-dataset topics (such as `mucca` with $1,866$ files) cannot be served by Case 2 Enhanced in a single pass without dynamically resizing the work plan.

9.2 Suggested Future Work

1. **Physical Multi-Host Deployment:** Deploy the binaries across three physical hardware machines connected via a Gigabit Ethernet switch to evaluate physical network latency, jitter, and link saturation.
2. **Pipelined Sliding-Window Acknowledgments:** In Case 2, replace synchronous stop-and-wait chunk acknowledgments with a sliding-window credit scheme. This would eliminate the loopback latency floor and allow Case 2 to match Case 1 speeds under zero-fault conditions.
3. **Dynamic Work-Stealing Coordinator:** Expand Case 2 Enhanced to dynamically allocate work units via a thread-safe task queue, eliminating the static `MAX_WORK_UNITS = 1024` ceiling and balancing load across heterogeneous network links.

10. Conclusion and References

10.1 Key Conclusions

This project successfully designed, implemented, audited, and benchmarked a complete POSIX TCP topic-based file distribution service. The empirical results validate the core architectural hypotheses:

1. **Concurrency is essential for throughput under load:** The multi-threaded server prevents queuing serialization, delivering a $4.08\times$ **throughput gain** (6.50 MB/s vs. 1.59 MB/s) under 8 concurrent clients.
2. **Session management guarantees fault resilience:** Checkpoint-based resumption completely eliminates redundant payload delivery (**0 bytes redundant payload** vs. 1.19 MB redundant in unmanaged restart at $p = 0.10$), trading a minor zero-fault latency overhead for rock-solid stability under adverse network conditions.
3. **Parallel connections accelerate file distribution:** Slicing transfers across multiple TCP connections achieves near-linear speedup (**$3.89\times$ for 4 connections**, 97.2% efficiency) on multi-file workloads while maintaining full backward compatibility with mandatory CLI specifications.

10.2 Formal Academic References

1. Postel, J. (1981). **Transmission Control Protocol - DARPA Internet Program Protocol Specification**. RFC 793 / STD 7. Internet Engineering Task Force.
2. Eddy, W. (2022). **Transmission Control Protocol (TCP) Specification**. RFC 9293. Internet Engineering Task Force.
3. Stevens, W. R., Fenner, B., & Rudoff, A. M. (2004). **UNIX Network Programming, Volume 1: The Sockets Networking API** (3rd ed.). Addison-Wesley Professional.
4. IEEE & The Open Group. (2018). **POSIX.1-2008 / IEEE Std 1003.1-2008 (Base Specifications, Issue 7)**. The Open Group Technical Standard.
5. Corrado, A. (2020). **Animals-10 Dataset: 10 Categories of Animals**. Kaggle Dataset Repository. `https://www.kaggle.com/datasets/alessiocorrado99/animals10`.
6. DA-IICT Course Faculty. (Autumn 2026). **IT305 Computer Networks Laboratory: Socket Programming Project Specification**. Department of ICT, DA-IICT.
7. Merged Groups 071 & 072. (September 2026). **Design Document: Fault-Tolerant Topic-Based File Distribution Service**. IT305 Project Archive, DA-IICT.